



CEUTEC

DESARROLLO DE APLICACIONES DE VANGUARDIA

TEMA: HTTP

TAREA 1

SECCIÓN 861

GIXEL ALEXANDER VARELA TRIMINIO

61551130

CATEDRÁTICO:

ING. ELVIN BOBADILLA

02/05/2026

San Pedro Sula

Introducción

El protocolo HTTP es uno de los pilares principales del funcionamiento de la Web y de muchas aplicaciones modernas. Cada vez que un navegador, una aplicación móvil o un sistema empresarial se comunica con un servidor web, normalmente utiliza solicitudes HTTP para pedir, enviar, modificar o eliminar información. Por esta razón, comprender los métodos HTTP es fundamental para el desarrollo de APIs y servicios web.

En esta tarea se investigan los métodos principales del protocolo HTTP y se desarrolla un prototipo práctico utilizando Node.js y Express. El objetivo es relacionar la teoría con la práctica, demostrando cómo cada método se aplica en una API básica. La investigación se apoya en documentación técnica reconocida, incluyendo MDN Web Docs, RFC 9110 y la documentación oficial de Express.

Objetivos

Objetivo general

Comprender el funcionamiento de los métodos principales del protocolo HTTP y aplicarlos mediante el desarrollo de una API básica utilizando Node.js y Express.

Objetivos específicos

1. Definir los métodos HTTP más utilizados en el desarrollo de aplicaciones web.
2. Explicar el propósito principal de cada método y su uso en sistemas reales.
3. Describir la estructura básica de una petición HTTP.
4. Identificar las diferencias principales entre los métodos HTTP.
5. Construir un prototipo funcional en Node.js y Express que implemente los métodos estudiados.

Marco teórico

HTTP, por sus siglas en inglés Hypertext Transfer Protocol, es un protocolo de comunicación utilizado para el intercambio de información entre clientes y servidores. El RFC 9110 define HTTP como un protocolo de nivel de aplicación, sin estado, utilizado para sistemas de información distribuidos, colaborativos e hipermedia (Fielding et al., 2022).

Los métodos HTTP indican la acción que se desea realizar sobre un recurso determinado. MDN Web Docs explica que cada método posee una semántica diferente y que algunos pueden clasificarse como seguros, idempotentes o almacenables en caché (MDN Web Docs, 2025).

En el desarrollo de APIs, estos métodos permiten organizar las operaciones de forma clara. Por ejemplo, GET se utiliza para consultar datos, POST para crear información, PUT para reemplazar recursos y DELETE para eliminar recursos.

Estructura básica de una petición HTTP

Una petición HTTP se compone principalmente de una línea de solicitud, encabezados y, cuando es necesario, un cuerpo. La estructura general puede representarse de la siguiente manera:

```
MÉTODO /ruta/recurso HTTP/1.1
Host: ejemplo.com
Content-Type: application/json

{
  "campo": "valor"
}
```

En esta estructura, el método indica la operación solicitada; la ruta indica el recurso; los encabezados contienen información adicional; y el cuerpo contiene datos enviados al servidor cuando la operación lo requiere, como ocurre normalmente con POST y PUT.

Investigación de los métodos HTTP principales

GET

Definición: GET solicita una representación de un recurso específico. Su función principal es obtener información sin modificar el estado del servidor.

Propósito principal: Consultar datos o recuperar información existente.

Ejemplo de uso en sistemas reales: Una tienda en línea utiliza GET para mostrar el listado de productos o consultar el detalle de un producto específico.

Estructura básica de una petición:

```
GET /api/productos HTTP/1.1
Host: localhost:3000
```

Diferencias principales: A diferencia de POST, PUT y DELETE, GET no debe modificar datos.

Es considerado seguro e idempotente.

POST

Definición: POST envía una entidad a un recurso específico, causando con frecuencia un cambio en el estado del servidor.

Propósito principal: Crear un nuevo recurso o enviar datos para ser procesados.

Ejemplo de uso en sistemas reales: Un sistema de matrícula usa POST para registrar un nuevo estudiante en la base de datos.

Estructura básica de una petición:

```
POST /api/productos HTTP/1.1
Host: localhost:3000
Content-Type: application/json

{"nombre":"Teclado","precio":45.99}
```

Diferencias principales: POST no es idempotente, porque enviar la misma petición varias veces puede crear varios registros.

PUT

Definición: PUT reemplaza completamente la representación actual de un recurso con los datos enviados en la petición.

Propósito principal: Actualizar o reemplazar un recurso completo.

Ejemplo de uso en sistemas reales: Un sistema de inventario usa PUT para reemplazar toda la información de un producto existente.

Estructura básica de una petición:

```
PUT /api/productos/1 HTTP/1.1
Host: localhost:3000
Content-Type: application/json

{"nombre":"Laptop","categoria":"Tecnología","precio":920}
```

Diferencias principales: PUT se diferencia porque reemplaza el recurso completo, mientras que PATCH modifica solo campos específicos.

DELETE

Definición: DELETE elimina un recurso específico en el servidor.

Propósito principal: Borrar información existente cuando ya no se necesita.

Ejemplo de uso en sistemas reales: Un sistema administrativo usa DELETE para eliminar un registro de producto discontinuado.

Estructura básica de una petición:

```
DELETE /api/productos/1 HTTP/1.1
Host: localhost:3000
```

Diferencias principales: DELETE modifica el estado del servidor. A diferencia de GET, no es un método seguro.

Comparación general de métodos HTTP

Método	Propósito	Modifica datos	Idempotente	Ejemplo
GET	Consultar	No	Sí	Listado o detalle de productos
POST	Crear o procesar	Sí	No	Registrar un nuevo producto
PUT	Reemplazar completo	Sí	Sí	Sustituir todos los datos de un producto
PATCH	Actualizar parcial	Sí	No necesariamente	Cambiar solo el precio o estado
DELETE	Eliminar	Sí	Sí	Borrar un producto por ID

Conclusiones

Los métodos HTTP son esenciales para el desarrollo de APIs porque permiten definir claramente qué acción se realizará sobre cada recurso. Comprender su propósito evita confusiones entre consultar, crear, reemplazar, modificar o eliminar información.

El prototipo desarrollado con Node.js y Express demuestra que la teoría puede aplicarse de manera directa en un servidor web funcional. Cada endpoint implementado representa una operación común utilizada en sistemas reales.

Finalmente, se concluye que usar correctamente los métodos HTTP mejora la organización, comprensión y mantenimiento de una API. Esta base es importante para desarrollar aplicaciones web modernas de forma profesional.

Enlace Repositorio Github

https://github.com/95alexandervarela/TareaNo.1_61551130.git

Bibliografía

Dusseault, L., & Snell, J. (2010). RFC 5789: PATCH Method for HTTP. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc5789>

Express. (s. f.). Routing. Express. <https://expressjs.com/en/guide/routing.html>

Fielding, R., Nottingham, M., & Reschke, J. (2022). RFC 9110: HTTP Semantics. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9110>

MDN Web Docs. (2025, 7 de julio). Métodos de petición HTTP. Mozilla. <https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Methods>